

P11 — Mini Evaluador de Expresiones Algebraicas

Fundamentos de Intérpretes y Evaluador de Expresiones

Integrantes del equipo:

Jose Luis Canchanya Ramos

Piero Mauro Morales Sinche

Aldair Edgard Gutierrez Mamani

Rodrigo Matias Ortiz Montoya

```
$ ./eval_math  
> sin(pi/2)  
= 1.000000  
> x = 5 + e  
= 7.718282  
> list
```

Descripción del Proyecto

¿Qué es?

Evaluator interactivo de expresiones algebraicas escrito en lenguaje C puro. Funciona como intérprete tipo REPL (Read-Eval-Print Loop) similar a GeoGebra. Construye un Árbol de Sintaxis Abstracta (AST) usando un Lexer y Parser de descenso recursivo desde cero, sin dependencias externas más allá de la biblioteca matemática estándar (-lm).

Herramientas

- ▶ GCC / Clang con -Wall -Wextra -lm
- ▶ GitHub con Makefile y CI automático
- ▶ Tests automáticos en cada push

6

Operadores

+ - * / ^ %

12+

Funciones

trig, log, rango

5

Módulos C

lexer, parser, eval...

Requerimientos Funcionales

RF1

Evaluación Aritmética

Operaciones: Suma +, Resta -, Multiplicación *, División /, Módulo %, Potencia ^

RF2

Soporte de Variables

Asignación y reutilización de variables ($a = x + y$) en toda la sesión interactiva

RF3

Funciones Matemáticas

exp, sqrt, pow | log, log2, log10 | sin, cos, tan, asin, acos, atan, atan2 | max, min, abs

RF4

Manejo de Excepciones

Validación de división por cero y errores de dominio sin colgar la aplicación

RF5

Configuración Angular

Cambio de unidad entre Radianes y Grados mediante comandos rad / deg

RF6

Entorno REPL

Ciclo Read-Eval-Print-Loop con comandos list (ver variables) y quit / exit

Requerimientos No Funcionales

◆ Restricción de Bibliotecas

- Solo ANSI C nativa
- `stdio.h`, `stdlib.h`, `string.h`, `math.h`
- Sin dependencias externas

⬡ Arquitectura Modular

- Separación limpia: Lexer, Parser, AST, Eval, CLI
- Fácilmente extensible y mantenible
- Sujeta a tests exhaustivos

Rendimiento

- Evaluación casi instantánea
- Overhead mínimo en C nativo, no hay mucha carga al equipo
- Ideal para máquinas con pocos recursos

Robustez y Seguridad

- Gestión de memory leaks con `free_ast()`, libera la memoria
- No crashea ante errores de dominio
- Banderas de error controladas (`eval_error`), similar a un try catch

Gramática Formal — EBNF

Texto del párrafo

```
Expression ::= Assign
Assign      ::= IDENTIFIER "=" Assign
            | AddSub
AddSub      ::= MulDivMod
            { ("+" | "-") MulDivMod }
MulDivMod   ::= Power
            { ("*" | "/" | "%") Power }
Power       ::= Unary { "^" Power }
Unary       ::= ("+" | "-") Unary | Primary
Primary     ::= NUMBER
            | "(" Assign ")"
            | IDENTIFIER
            | IDENTIFIER "("
              [ Assign { "," Assign } ] ")"
```

Notas de Precedencia

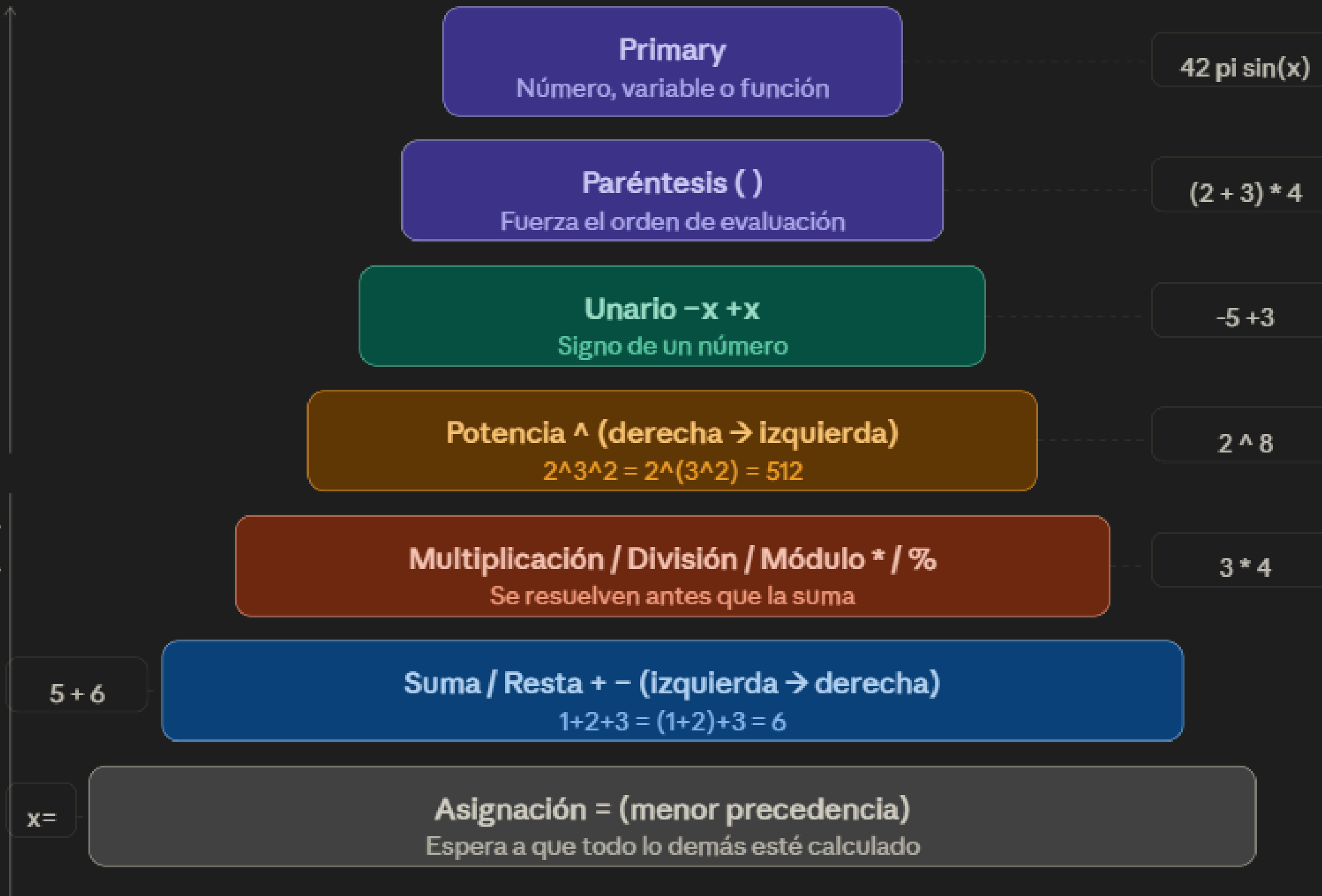
↑ Mayor prec.	Potencia (^)
Media prec.	* / %
Media prec.	+ -
↓ Menor prec.	Asignación (=)
Asociatividad	Potencia y = → derecha
Unary	- y + aplicados antes de Primary

Pirámide completa — de mayor a menor precedencia

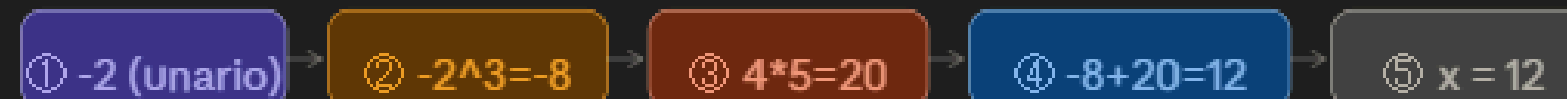
...

Lo que está más arriba se resuelve PRIMERO

mayor precedencia →



Ejemplo completo: $x = -2^3 + 4 * 5$



Gramática EBNF y Pipeline de Procesamiento

Gramática Formal (EBNF)

```
Expression ::= Assign
Assign      ::= IDENT '=' Assign | AddSub
AddSub      ::= MulDivMod { ('+' | '-') MulDivMod }
MulDivMod   ::= Power { ('*' | '/' | '%') Power }
Power       ::= Unary { '^' Power }
Unary       ::= ('+' | '-') Unary | Primary
Primary     ::= NUMBER
              | '(' Assign ')'
              | IDENTIFIER
              | IDENTIFIER '(' [Assign
                           { ',' Assign }] ')'
```



Precedencia de Operadores (mayor → menor)

Paréntesis ()

Unario -x +x

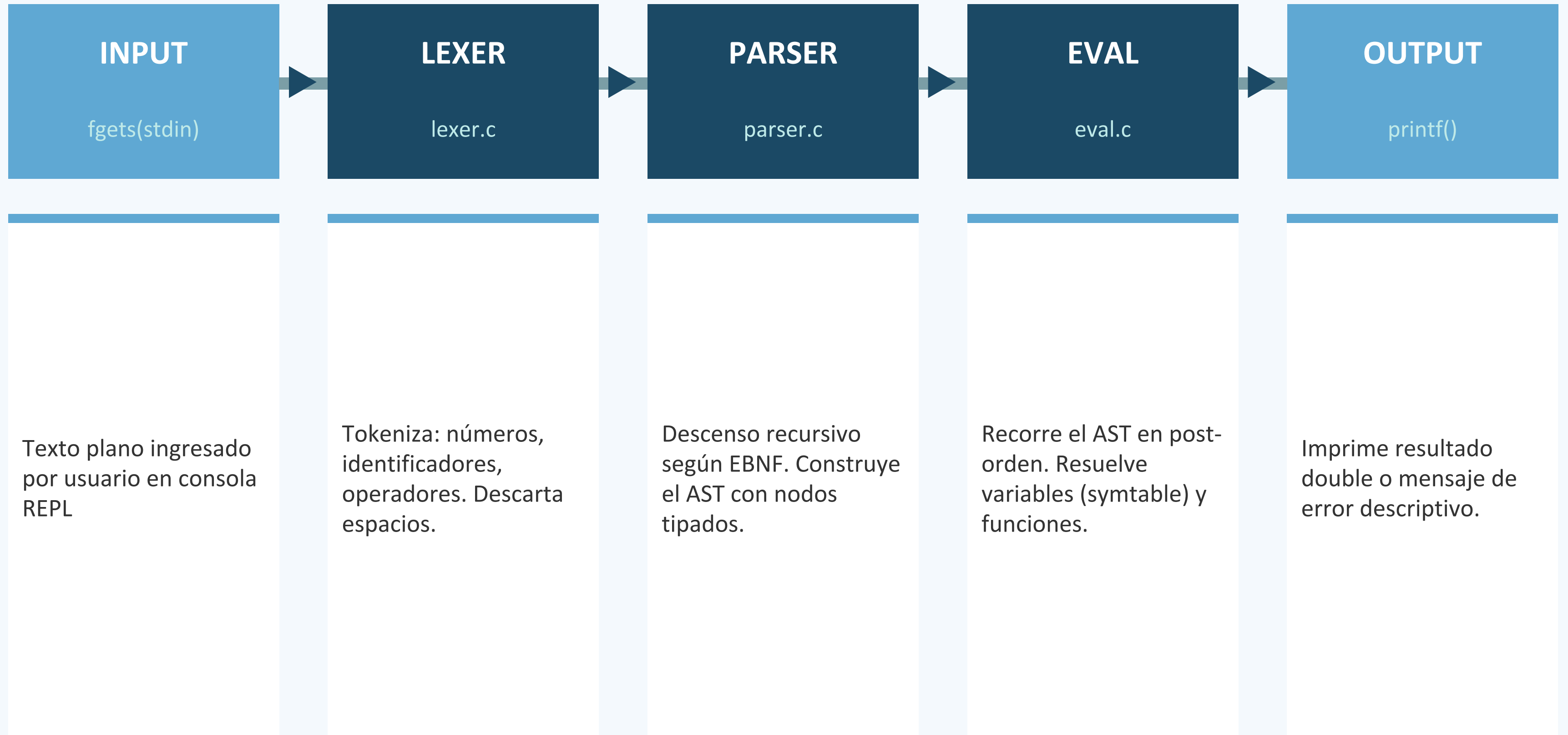
Potencia ^ (derecha→izq)

Mul / Div / Mod * / %

Suma / Resta + -

Asignación =

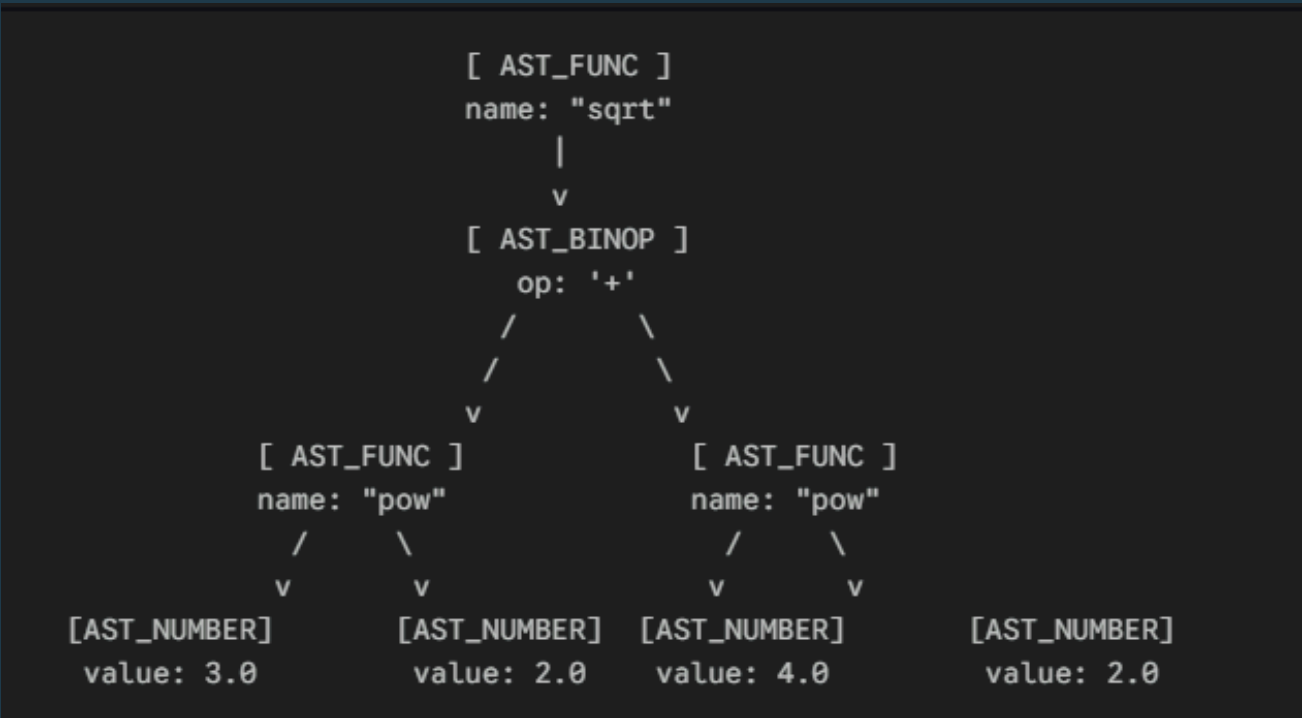
Pipeline: Lexer → Parser → Evaluador



Modelo de Datos — Estructuras C

struct ASTNode (Árbol de Sintaxis Abstracta)

```
ASTNodeType type;
union {
    double value;
    TokenType op;
};
char name[32];
ASTNode* left;
ASTNode* right;
ASTNode** args;
int num_args;
```



struct Symbol (x100)

```
char name[32];
double value;
```

Arreglo: symbol_table (Capacidad: 100 elementos)

Índice	name (char[32])	value (double)
0	"pi\0\0\0\0\0\0\0\0\0\0..."	3.141592653589793
1	"e\0\0\0\0\0\0\0\0\0\0..."	2.718281828459045
2	"a\0\0\0\0\0\0\0\0\0\0..."	5.500000
3	[Basura residual en memoria]	[Basura residual]
...	...	...
99	[Basura residual en memoria]	[Basura residual]

FuncEntry (callback table)

```
char* name;
int num_args;
MathFunc1 fn1; // f(x)
MathFunc2 fn2; // f(x,y)
```

Modelo de Datos — El Nodo del AST (ASTNode)

Gestión Eficiente (Union)

Ahorra memoria compartiendo el mismo espacio para almacenar o un literal (double value) o un operador (TokenType op).

Soporte Multi-argumento

Memoria dinámica con `**args` (sub arbol) y `num_args`(número de argumentos) para funciones complejas (como `max` o `atan2`).

Atributos de Texto

Un arreglo de tamaño fijo `char name[32]` reservado para nombres de variables o funciones.

Punteros de Recursión

Los enlaces `left` y `right` que permiten construir el árbol binario hacia abajo.

Tipología del Nodo (ASTNodeType)

- Hojas (Datos sin hijos):
 - AST_NUMBER: Contiene un valor flotante directo.
 - AST_VAR: Contiene el nombre de una variable para buscarla en la tabla de símbolos.
- Ramas (Operaciones con hijos):
 - AST_BINOP: Operador binario (ej. suma, resta) que procesa `left op right`.
 - AST_UNOP: Operador unario (ej. signo negativo) que procesa `op left`.
 - AST_FUNC: Llamada a función que ejecuta un nombre usando el arreglo de `args[]`.
 - AST_ASSIGN: Operación de guardado de estado (`nombre = right`).

Tabla de Símbolos — Memoria de Estado y Variables

Arreglo Estático

Implementada utilizando un arreglo global de tamaño fijo (`symbol_table`) con un límite máximo de 100 elementos (`MAX_VARIABLES`).

Puntero de Estado

Utiliza un contador interno (`symbol_count`) para llevar el registro exacto de cuántas variables están activas en la sesión actual

Composición del Símbolo

Cada ranura almacena una estructura `Symbol` que contiene un identificador de texto (`char name[32]`) y su valor numérico asociado (`double value`).

Escritura Inteligente

Al procesar un nodo de asignación (ej. `a = 5`), el sistema primero busca si la variable ya existe para actualizarla; si no, la agrega en la siguiente posición libre.

Funciones Symtable:

- `symtable_init()`: Se invoca una única vez al arrancar el programa, prepara la tabla para su uso e inserta internamente las dos constantes matemáticas predefinidas (`pi` y `e`) llamando a su vez a `symtable_set`.
- `symtable_list()`: Esto ocurre cuando el usuario teclea exactamente el comando `list` en la consola, se llama desde el `main`.
- `symtable_get()`: Cuando el evaluador está recorriendo el árbol y se topa con una variable (como `a`), detiene el cálculo momentáneamente e invoca `symtable_get(node->name, &val)` para buscar cuánto vale `a`.
- `symtable_set()`: Cuando el usuario escribe una asignación como `x = 5 + 3`, el evaluador primero resuelve el lado derecho (que da 8). Inmediatamente después, llama a `symtable_set(node->name, val)` para inyectar o actualizar ese número 8 bajo el nombre `x` dentro del arreglo global.

Tabla de Callbacks (FuncEntry) — Extensibilidad sin if/else

La Estructura de Enrutamiento

- El Molde (FuncEntry): Es una estructura que asocia un nombre de texto ingresado por el usuario con la dirección de memoria de la función matemática real que debe ejecutar el procesador.
- Atributos de Identificación: Contiene char name[16] para el texto (ej. "sin", "max") y int num_args para validar cuántos parámetros requiere (1 o 2).
- Punteros a Función: Utiliza los tipos personalizados MathFunc1 (para funciones de un argumento como sin(x)) y MathFunc2 (para funciones de dos argumentos como max(a, b)).

```
typedef struct {  
    char name[16];  
    int num_args;  
    MathFunc1 func1;  
    MathFunc2 func2;  
} FuncEntry;
```

Tabla de Callbacks (FuncEntry) — Extensibilidad sin if/else

La Tabla Estática (func_table)

- En lugar de tener una función gigante con 16 bloques if / else if comparando strings, el sistema utiliza un arreglo global func_table[].
- Cada ranura del arreglo es un FuncEntry que empareja el texto con la función nativa de <math.h> o con funciones "wrapper" personalizadas (ej. f_sin, f_max).
- Extensibilidad Máxima: Para agregar una nueva función al evaluador, no se necesita reescribir la lógica del sistema; basta con agregar una sola línea en este arreglo.

```
static FuncEntry func_table[] = {
    {"sin", 1, f_sin, NULL},
    {"cos", 1, f_cos, NULL},
    {"tan", 1, f_tan, NULL},
    {"asin", 1, f_asin, NULL},
    {"acos", 1, f_acos, NULL},
    {"atan", 1, f_atan, NULL},
    {"atan2", 2, NULL, f_atan2},
    {"log", 1, f_log, NULL},
    {"log2", 1, f_log2, NULL},
    {"log10", 1, f_log10, NULL},
    {"exp", 1, exp, NULL},
    {"sqrt", 1, f_sqrt, NULL},
    {"abs", 1, fabs, NULL},
    {"max", 2, NULL, f_max},
    {"min", 2, NULL, f_min},
    {"pow", 2, NULL, pow}
};
```


Tabla de Callbacks (FuncEntry) — Extensibilidad sin if/else

Ejecución Segura (call_function)

- Cuando el AST necesita evaluar una función, el módulo eval.c delega el trabajo a call_function.
- Esta función busca el nombre en la tabla, verifica que el número de argumentos coincida exactamente (func_table[i].num_args != num_args), y luego ejecuta la matemática llamando al puntero correspondiente (func1 o func2)

```
static int num_functions = sizeof(func_table) / sizeof(func_table[0]);

int call_function(const char* name, const double* args, int num_args, double* out_result) {
    for (int i = 0; i < num_functions; i++) {
        if (strcmp(func_table[i].name, name) == 0) {

            if (func_table[i].num_args != num_args) {
                printf("Error: '%s' espera %d argumento(s), pero se dieron %d.\n", name, func_table[i].num_args, num_args);
                return 0; // Error de invocacion
            }

            if (num_args == 1) {
                *out_result = func_table[i].func1(args[0]);
            } else if (num_args == 2) {
                *out_result = func_table[i].func2(args[0], args[1]);
            }
            return 1; // Exitoso
        }
    }

    printf("Error: Función desconocida '%s'\n", name);
    return 0;
}
```

Casos de Uso del Sistema

CU-01 — Usuario

Evaluar expresión aritmética

Ingresa expresión, el sistema tokeniza, parsea y retorna el resultado numérico.

CU-02 — Usuario

Definir variable

Asigna un valor o expresión a una variable para reutilizar en la sesión.

CU-03 — Usuario

Llamar función matemática

Invoca funciones como `sin()`, `log()`, `max()` con argumentos evaluados.

CU-04 — Usuario

Cambiar modo angular

Alternar entre radianes y grados con comandos `rad` / `deg`.

CU-05 — Sistema

Reportar error descriptivo

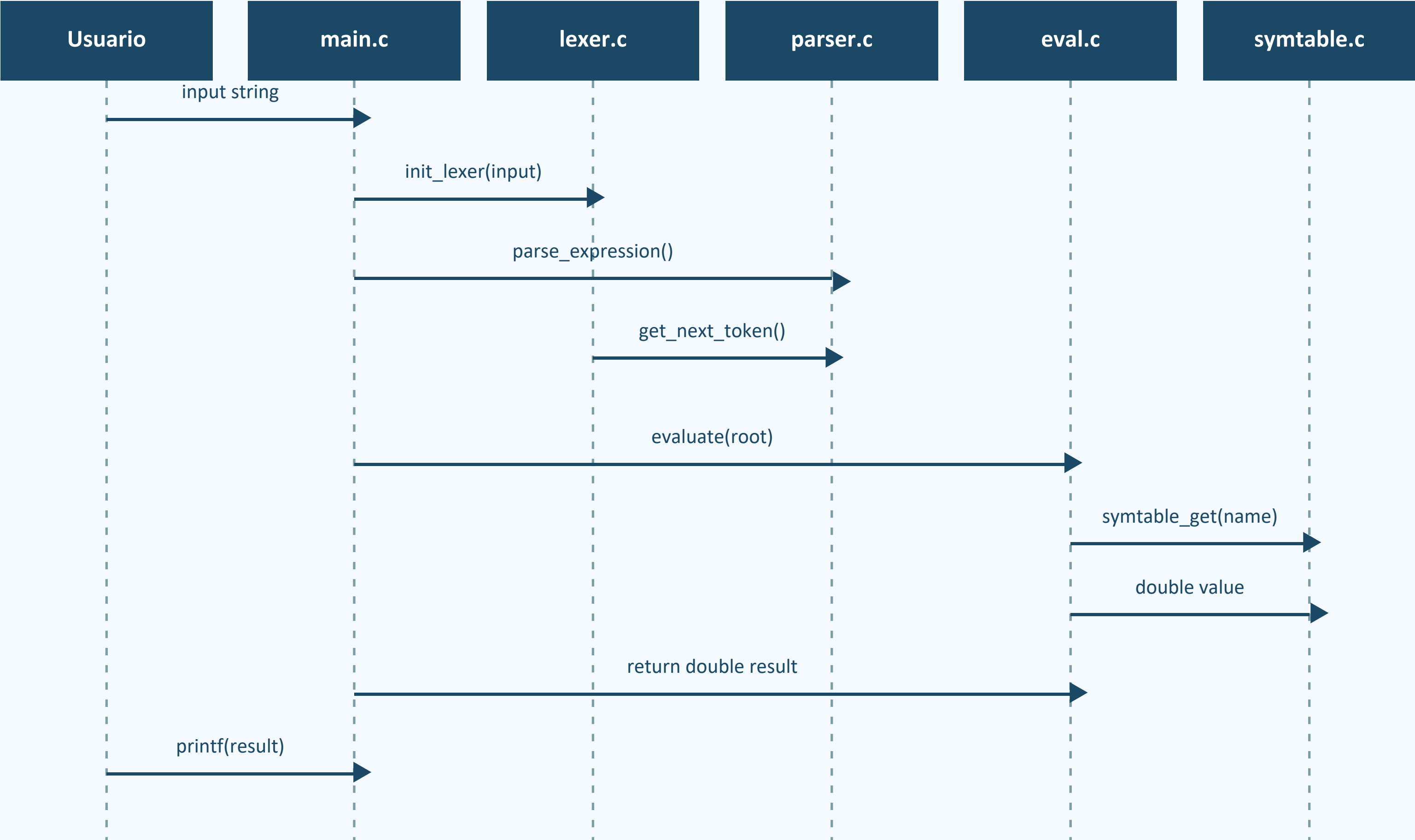
Detecta y comunica errores de dominio, sintaxis o división por cero sin crashear.

CU-06 — Usuario

Listar variables guardadas

Comando `list` muestra el estado de la tabla de símbolos en tiempo real.

Diagrama de Secuencia — Evaluar Expresión



Interfaz de Usuario — REPL de Consola

● ● ● eval_math – bash

```
=====
Mini Evaluador de Expresiones Algebraicas (P11)
=====
Escribe una expresión (ej. a = 5 * sin(pi/2))
Comandos especiales:
list      - Muestra las variables definidas
quit      - Sale del programa
deg       - Cambia el modo a Grados
rad       - Cambia el modo a Radianes (por defecto)
=====
> sin(pi/2)
= 1.000000
> x = 3.5 + pi
= 6.641593
> deg
[Modo: Grados]
> sin(90)
= 1.000000
> log(-5)
Error: log de negativo
> list
x = 6.641593
```

Comandos REPL	
list	Ver variables guardadas
quit / exit	Salir del evaluador
rad	Cambiar a Radianes
deg	Cambiar a Grados
a = expr	Asignar variable
sin(x)	Funciones matemáticas
max(a,b)	Funciones multi-arg

Casos de Prueba

Entrada	Salida Esperada	Categoría
5 + 3 * 2	11	Precedencia
2 ^ 3 ^ 2	512	Asociatividad →
log(e)	1	Constantes
sin(pi/2)	1	Trigonometría rad
sin(90) [modo deg]	1	Modo grados
max(10-2, 5*2)	10	Funciones multi-arg
sqrt(pow(3,2)+pow(4,2))	5	Composición
10.5 % 3	1.5	Módulo flotante
-5 + 10	5	Unary minus
log(-5)	Error dominio	Error esperado
5 / 0	Error div/0	Error esperado

¡Gracias!

P11 — Mini Evaluador de Expresiones Algebraicas

Entregable 1 — Análisis y Diseño | Fundamentos de Intérpretes

GitHub: github.com/felizmedioa/Proy_LP1_2026_1